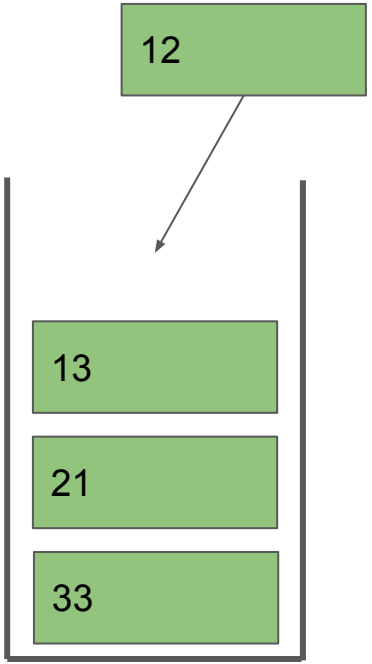
Java Collections Stream API

Содержание

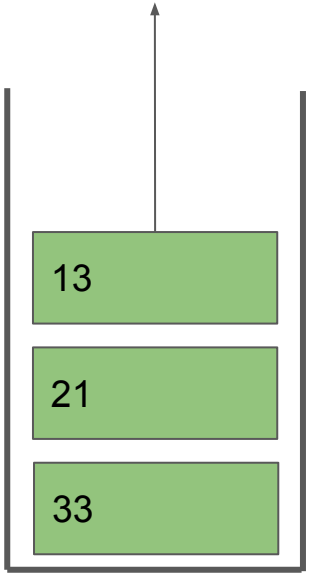
- Коллекции и словари
  - виды коллекций и словарей
  - иерархия интерфейсов и классов
  - методы
  - класс Collections
- Stream API
  - промежуточные операции
  - терминальные операции
  - класс Collectors
- Домашнее задание 🎉

Стек

Методы - pop(), push(element), isEmpty()

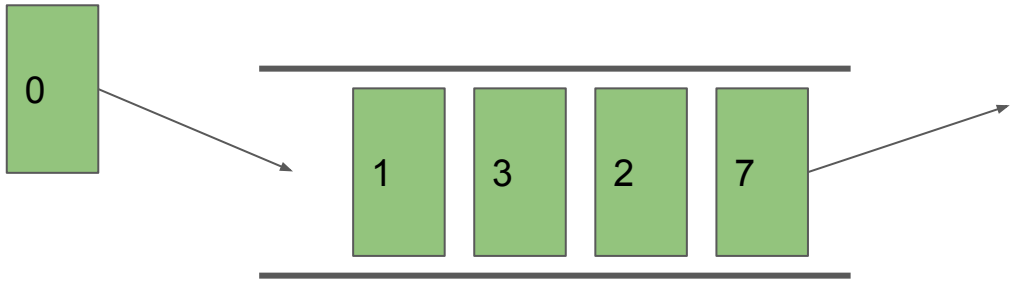


LIFO - Last in, first out



Очередь

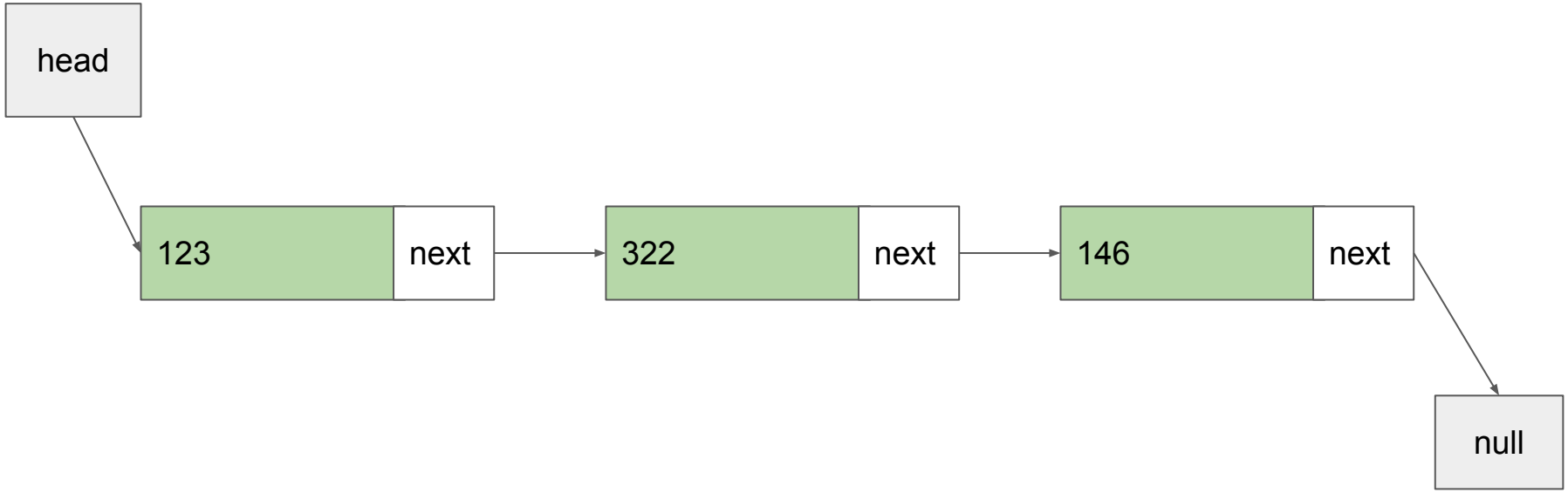
Методы - pop(), push(element), isEmpty()



FIFO - First in, first out

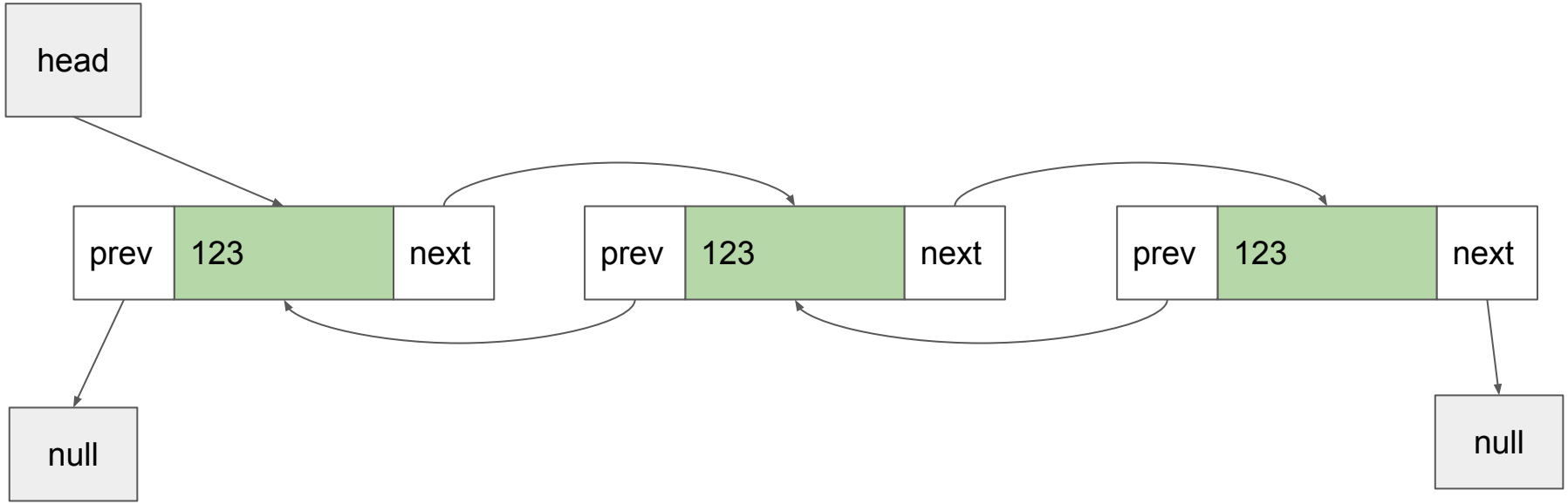
Односвязный список

Методы addAfter(element, newElement), removeAfter(element)

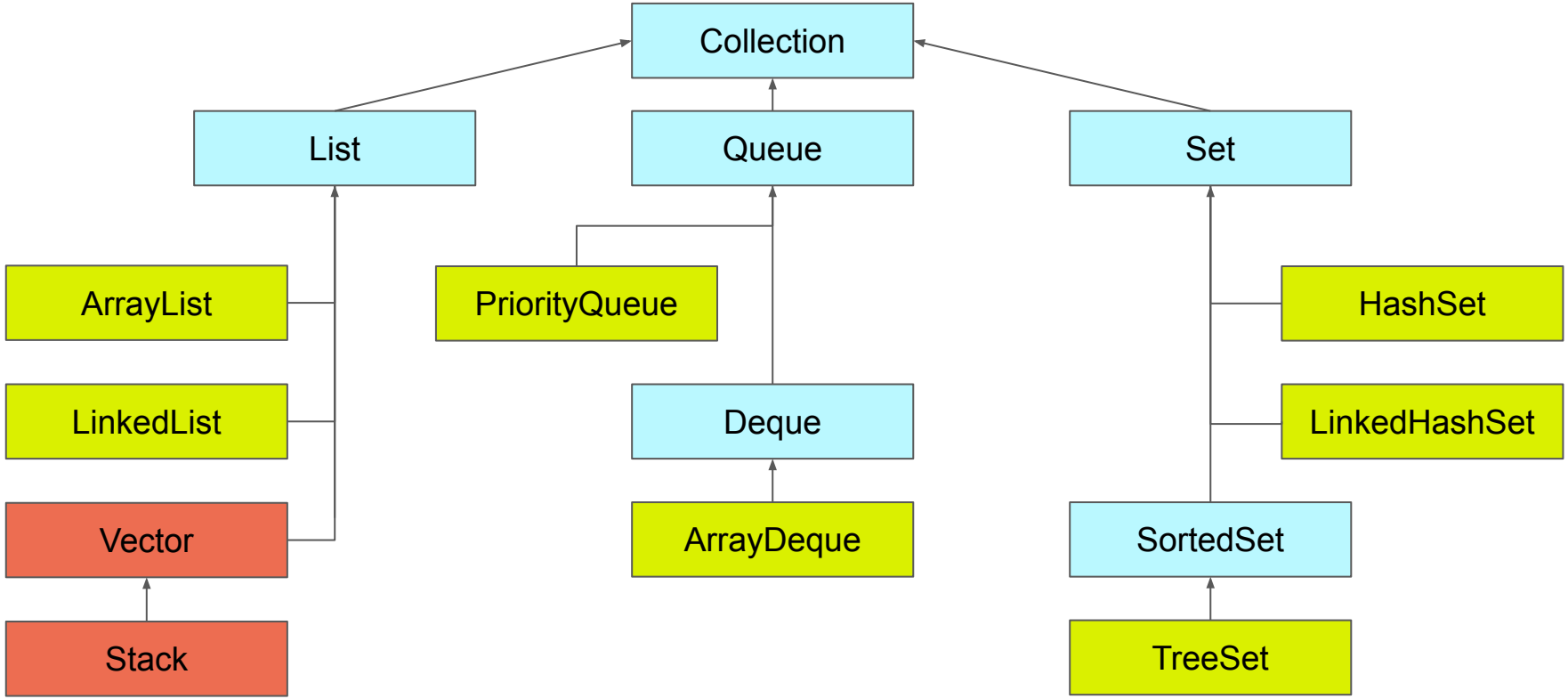


Двусвязный список

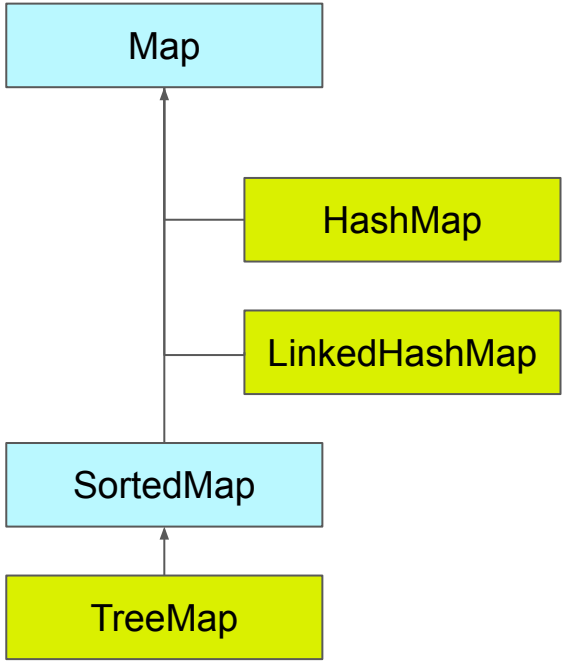
Методы addAfter, addBefore, remove



Иерархия коллекций



Иерархия коллекций



Интерфейсы

Collection - абстрактная коллекция элементов. Содержит методы `add(obj), addAll(collection), remove(obj), contains(obj), iterator()`

List - Упорядоченная коллекция, допускающая дублирование элементов. Дополнительно содержит методы получения и вставки элементов по индексу `get(index), set(index, obj), remove(index)`

Set - множество элементов, добавляет уникальность
SortedSet - добавляет порядок элементам и методы `first(), last()`

Queue и **Deque** - односторонняя и двусторонняя очереди. Вставка и получения элементов осуществляются с концов

Collection

Применяйте Collection, если например вам нужно только проитерироваться по элементам (порядок неважен)

```java
public void publishVacancies(Collection<Integer> vacancyIds) {
    for(Integer id : vacancyIds) {
        publishVacancy(id);
    }
}

public void publishVacancies(Collection<Integer> vacancyIds) {
    Iterator<Integer> it = vacancyIds.iterator();
    while (it.hasNext()) {
        publishVacancy(it.next());
    }
}
```

Реализации List

ArrayList - реализация List, основанная на массиве.
Изначальный размер массива по умолчанию равен 10.
Можно указать свой initialCapacity.
При заполнении массива выделяет новый большего размера и копирует данные из старого

```
List<Integer> list1 = new ArrayList<>();
// Произойдет выделение памяти под массив
// в 10 элементов при добавлении
list1.add(1);

List<Integer> list2 = new ArrayList<>(100);
// Массив из 100 элементов создан в конструкторе
list2.add(1);
```

Реализации List

LinkedList - двусвязный список. Каждый элемент имеет ссылку на предыдущий и следующий. Реализует также интерфейс Queue и Deque

Vector - старая синхронизированная реализация, методами не отличающаяся от ArrayList. Не рекомендуется использовать

Map и SortedMap

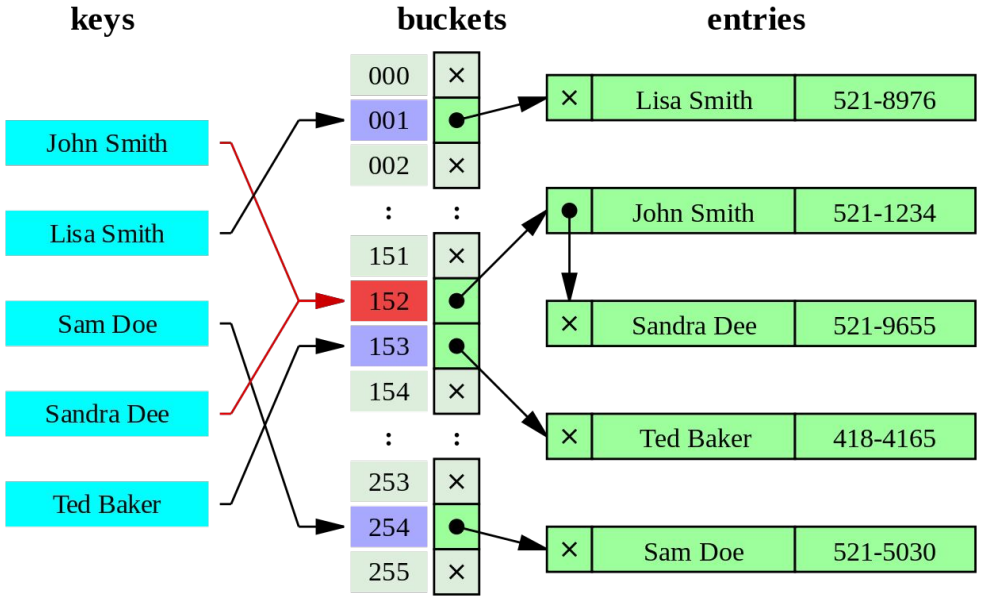
Map - ассоциативный массив или словарь, хранящий пары “ключ-значение”. Ключи уникальны в пределах одного словаря. Есть методы вставки, получения и удаления элемента по ключу.

```
Map<Integer, Vacancy> vacancies;
vacancies.put(vacancy.getId(), vacancy);
vacancies.get(vacancy.getId());
vacancies.remove(vacancy.getId());
```

SortedMap - разновидность Map с отсортированными ключами
В дополнении имеет методы firstKey() и lastKey() - наименьший и наибольший ключ

Реализации Map

HashMap - словарь на основе хеш-таблицы. Различные ключи могут иметь одинаковые хэши, в этом случае используются связанные списки в ячейках.



Реализации Map

При создании можно указать начальный capacity

```
new HashMap<>(int capacity);
```

А так же loadFactor

```
new HashMap<>(int capacity, float loadFactor);
```

HashMap

Для вычисления хэша и проверки на равенство используются hashCode() и equals()

Время вставки, поиска и удаления - O(1)

Ключ может быть null

```
Map<Integer, String> map =
  new HashMap<>();
map.put(1, “Value”);
map.put(2, “Value”);
```

HashCode

По умолчанию - зависит от адреса объекта при первом вызове (в большинстве реализаций JVM)

```
@Override
public int hashCode() {
  return Objects.hash(id, lastName, email);
}
```

Реализации Map

LinkedHashMap - интерфейсно не отличается от Map, но при итерировании по ключам или значениям гарантирует порядок элементов по их добавлению в коллекцию (или против, в зависимости от параметра accessOrder)

Для этого кроме обычной хэш-таблицы есть связанный список, который хранит порядок добавления. Асимптотика операций остается O(1), но при этом константа по времени и памяти несколько больше

Реализации Map

TreeMap - словарь на основе красно-черного дерева. Узлы таким образом можно обходить в отсортированном порядке. Операции вставки, удаления и поиска выполняются за O(log n)

```
Map<Integer, String> map = new TreeMap<>();
map.put(5, "vacancy 1");
map.put(3, "vacancy 2");
map.put(4, "vacancy 3");
map.forEach((key, value) -> {
      System.out.print(key);
      System.out.print(" - ");
      System.out.println(value);
});
```

```
//Вывод
3 - vacancy 2
4 - vacancy 3
5 - vacancy 1
```

Реализации Set

HashSet - множество на основе хэш таблицы. Реализовано с помощью HashMap, где ключи это добавляемые объекты, а значениями является просто фиктивный (константный) new Object()

Добавление, удаление, поиск осуществляются за O(1)

LinkedHashSet - аналогичен HashSet, но позволяет обходить ключи в порядке их добавления. Как и LinkedHashMap используется дополнительный связанный список

Реализации Set

TreeSet - множество на основе красно-черного дерева. Узлы таким образом можно обходить в отсортированном порядке. Операции вставки, удаления и поиска выполняются за O(log n)

```
Set<Integer> set = new TreeSet<>();
set.add(5);
set.add(3);
set.add(4);
set.forEach(System.out::println);
// 3 4 5
```

Set - резюме

Все реализации Set по факту являются обертками над аналогичными реализациями Map и используют фиктивные значения для хранения объектов как ключей

Итого

	Время поиска по индексу	Время поиска по значению	Время вставки в начало	Время вставки в конец	Время вставки в середину	Поиск наименьшего элемента
ArrayList	O(1)	O(n)	O(n)	O(1) амортизационная	O(n)	O(n)
LinkedList	O(n)	O(n)	O(1)	O(1)	O(1)	O(n)
ArrayDeque	O(n)	O(n)	O(1)	O(1)	-	O(n)
PriorityQueue	-	O(n)	-	-	O(log n)	O(1)
HashMap/ LinkedHashMap	O(n)	O(1)	-	-	O(1)	O(n)
TreeMap	-	O(log n)	-	-	O(log n)	O(log n)

Создание коллекций

```
List<Integer> list = List.of(1, 2, 3);
// неизменяемая коллекция

Set<Integer> set = Set.of(1, 2, 3);
// неименяемая коллекция

List<Integer> list = Arrays.asList(1, 2, 3);
// можно изменять элементы, удалять и добавлять нельзя

new ArrayList<>(List.of(1, 2, 3));
// изменяемый список

Map<Integer, Integer> map = Map.of(1, 2, 3, 4);
// 1 -> 2, 3 -> 4
```

ВОПРОСЫ

ПЕРЕРЫВ

Класс java.util.Collections

Класс java.util.Collections

- Сортировка и поиск

```
void sort(List<T>)
void sort(List<T>, Comparator<? super T>)
int binarySearch(List<? extends Comparable<? super T>>, T)
int binarySearch(List<? extends T>, T, Comparator<? super T>)
```

```
List<Integer> list = new ArrayList<>(List.of(4, 1, 3, 2));
Collections.sort(list);
System.out.println(Collections.binarySearch(list, 2));
```

Класс java.util.Collections

- Вспомогательные методы

```
copy(List<? super T> dest, List<? extends T> src)
```

копирует элементы одного списка в другой

```
fill(List<? super T> list, T obj)
```

заполняет список заданным объектом

```
frequency(Collection<?> c, Object o)
```

возвращает кол-во вхождений данного объекта в коллекцию

```
reverse(List<?> list)
```

обращает порядок элементов в списке

Класс java.util.Collections

- Пустые неизменяемые коллекции

```
List<T> emptyList()
Set<T> emptySet()
Map<T, R> emptyMap()
SortedSet<T> emptySortedSet()
SortedMap<T, R> emptySortedMap()
```

- Неизменяемые коллекции, состоящие из одного элемента

```
Set<T> singleton(T t)
List<T> singletonList(T t)
Map<T, R> singletonMap(T t, R r)
```

Класс java.util.Collections

- Преобразование в неизменяемые коллекции

```
List<Integer> list = Collections.unmodifiableList(otherList);

list.add(1) // raise Exception

Collections.unmodifiableCollection(collection);
Collections.unmodifiableSet(set);
Collections.unmodifiableMap(map);
Collections.unmodifiableSortedSet(sortedSet);
Collections.unmodifiableSortedMap(sortedMap);
```

Класс java.util.Collections

- Преобразование в синхронизированные коллекции

```
List<Integer> list = Collections.synchronizedList(otherList);

for (int i : range) {
    CompletableFuture.runAsync(() -> list.add(i));
}

Collections.synchronizedCollection(collection);
Collections.synchronizedSet(set);
Collections.synchronizedMap(map);
Collections.synchronizedSortedSet(sortedSet);
Collections.synchronizedSortedMap(sortedMap);
```

Stream API

Stream API

Stream API это новый способ работать со структурами данных в функциональном стиле. Чаще всего с помощью stream в Java работают с коллекциями, но на самом деле этот механизм может использоваться для самых различных данных.

Позволяет применить функциональный стиль

Stream API

```
List<Vacancy> filteredVacancies = new ArrayList<>();
for (Vacancy vacancy : vacancies) {
  if (vacancy.getStatus() == Status.active && vacancy.getAreaIds().contains(areaId)) {
    filteredVacancies.add(vacancy);
  }
}
filteredVacancies.sort(Comparator.comparing(Vacancy::getTitle));
```

```
List<Vacancy> filteredVacancies = vacancies.stream()
  .filter(vacancy -> vacancy.getStatus() == Status.active)
  .filter(vacancy -> vacancy.getAreaIds().contains(areaId))
  .sorted(Comparator.comparing(Vacancy::getTitle))
  .collect(Collectors.toList());
```

Stream API

Применяемые к стриму методы можно разделить на две группы:

- Промежуточные (intermediate) — обрабатывают поступающие элементы и возвращают стрим. Промежуточных операторов в цепочке обработки элементов может быть много.

- Терминальные (terminal) — обрабатывают элементы и завершают работу стрима, так что терминальный оператор в цепочке может быть только один.

Создание Stream

- Пустой стрим: `Stream.empty()`
- Стрим из коллекции: `list.stream()`
- Стрим из Map: `map.entrySet().stream()`
- Стрим из массива: `Arrays.stream(array)`
- Стрим из указанных элементов: `Stream.of("a", "b", "c")`

Параллельные стримы

`parallel()` - делает стрим параллельным. Также можно вызвать у коллекции `parallelStream()`

`unordered()` - “забывает” об упорядоченности

Терминальные операторы

`void forEach(Consumer action)` - выполняет указанное действие для каждого элемента стрима.

`void forEachOrdered(Consumer action)` - тоже выполняет указанное действие для каждого элемента стрима, но перед этим добивается правильного порядка вхождения элементов. Используется для параллельных стримов, когда нужно получить правильную последовательность элементов.

Терминальные операторы

`Optional min(Comparator comparator)`

`Optional max(Comparator comparator)`

Поиск минимального/максимального элемента, основываясь на переданном компараторе.

```
int min = Stream.of(20, 11, 45, 78, 13)
    .min(Integer::compare).get();

int max = Stream.of(20, 11, 45, 78, 13)
    .max(Integer::compare).get();
```

Терминальные операторы

`Optional findAny()` - возвращает первый попавшийся элемент стрима. В параллельных стримах это может быть действительно любой элемент, который лежал в разбитой части последовательности.

`Optional findFirst()` - гарантированно возвращает первый элемент стрима, даже если стрим параллельный.

Если нужен любой элемент, то для параллельных стримов быстрее будет работать `findAny()`.

Терминальные операторы

```
Object[] toArray()
T[] toArray(IntFunction<T[]> func)
```

Собирает элементы стрима в массив

Например:

```
Integer[] array = Stream.of(1, 2, 3).toArray(Integer[]::new)
```

Терминальные операторы

`R collect(Collector collector)`. С его помощью можно собрать все элементы в список, множество или другую коллекцию, сгруппировать элементы по какому-нибудь критерию, объединить всё в строку и т.д.

```
List<Integer> list = Stream.of(1, 2, 3)
  .collect(Collectors.toList());
// list: [1, 2, 3]

String s = Stream.of("1", "2", "3")
  .collect(Collectors.joining("-", "<", ">"));
// s: "<1-2-3>"
```

Методы Collectors

`toList()` - собирает элементы в ArrayList.

`toSet()` - собирает элементы в HashSet.

`toCollection(Supplier collectionFactory)` - собирает элементы в заданную коллекцию. Если нужно конкретно указать, какой List, Set или другую коллекцию мы хотим использовать, то этот метод поможет.

```
Deque<Integer> deque = Stream.of(1, 2, 3, 4, 5)
    .collect(Collectors.toCollection(ArrayDeque::new));

Set<Integer> set = Stream.of(1, 2, 3, 4, 5)
    .collect(Collectors.toCollection(LinkedHashSet::new));
```

Методы Collectors

`toMap(Function keyMapper, Function valueMapper)`- собирает элементы в Map. Каждый элемент преобразуется в ключ и в значение, основываясь на результате функций keyMapper и valueMapper соответственно.

```
Map<Integer, Vacancy> vacanciesMap = vacancies.stream()
  .collect(Collectors.toMap(
      Vacancy::getId,
      Function.identity()
  ));
```

Методы Collectors

`toMap(Function keyMapper, Function valueMapper, BiFunction mergeFunction)` - версия с функцией мержа

```
Map<Integer, Integer> map = Stream.of(1, 1, 2, 3)
  .map(Collectors.toMap(x -> x, y -> y * y, (a, b) -> a);
// 1 -> 1, 2 -> 4, 3 -> 9

// использование Collectors.toMap(x -> x, y -> y * y) выдаст ошибку
```

Методы Collectors

```
groupingBy(Function classifier)
groupingBy(Function classifier, Collector downstream)
groupingBy(Function classifier, Supplier mapFactory, Collector downstream)
```

Группирует элементы по критерию, сохраняя результат в Map.

```
Map<Integer, List<String>> map = Stream.of(
    "ab", "c", "def", "gh", "ijk", "l", "mnop")
    .collect(Collectors.groupingBy(String::length));
map.entrySet().forEach(System.out::println);
// 1=[c, l]
// 2=[ab, gh]
// 3=[def, ijk]
// 4=[mnop]
```

Промежуточные операторы

`filter(Predicate predicate)` - фильтрует стрим, принимая только те элементы, которые удовлетворяют заданному условию

```
Stream.of(1, 2, 3)
  .filter(x -> x == 10)
  .forEach(System.out::print);
// Пустой вывод

vacancies.stream()
  .filter(Vacancy::hasRecruiter)
  .forEach(System.out::println);
```

Промежуточные операторы

`map(Function mapper)` - применяет функцию к каждому элементу и затем возвращает стрим, в котором элементами будут результаты функции. map можно применять для изменения типа элементов.

```
Stream.of("3", "4", "5")
    .map(Integer::parseInt)
    .map(x -> x + 10)
    .forEach(System.out::println);
// 13, 14, 15

vacancies.stream()
    .map(Vacancy::getTitle)
    .forEach(System.out::println);
// Выведет названия вакансий
```

Промежуточные операторы

`limit(long maxSize)` - ограничивает стрим maxSize элементами.

`skip(long n)` - пропускает n элементов стрима.

```
Stream.of(120, 410, 85, 32, 314, 12)
  .limit(4)
  .forEach(System.out::println);
// 120, 410, 85, 32

Stream.of(120, 410, 85, 32, 314, 12)
  .skip(2)
  .forEach(System.out::println);
// 85, 32, 314, 12
```

Промежуточные операторы

`distinct()` - возвращает различные элементы стрима.

`sorted(), sorted(Comparator<T> c)` - возвращает отсортированный стрим

```
persons.stream()
  .filtered(Person::hasEmail)
  .sorted(Comparator.comparing(Person::getName))
  .collect(Collectors.toList());
```

Промежуточные операторы

`flatMap(Function<T, Stream<R>> mapper)` - работает как map, но с одним отличием — можно преобразовать один элемент в ноль, один или множество других.

Получение всевозможные id ареек у вакансий

```
vacancies.stream()
  .map(Vacancy::getAreas)
//Stream<Set<Integer>>
  .distinct()
```

- не совсем то что нужно

```
class Vacancy {
  private Set<Integer> areas;
  public Str<Integer> getAreas() {
    return areas;
  }
}
```

Промежуточные операторы

`flatMap(Function<T, Stream<R>> mapper)` - работает как map, но с одним отличием — можно преобразовать один элемент в ноль, один или множество других.

Получение всевозможные id ареек у вакансий

```
vacancies.stream()
  .flatMap(vacancy -> vacancy.getAreas().stream())
//Stream<Integer>
  .distinct()
```

```
class Vacancy {
  private Set<Integer> areas;
  public Str<Integer> getAreas() {
    return areas;
  }
}
```

Stream нужен (или нет)

Когда Stream не нужен

Как не нужно

```
Set<Integer> set = list.stream().collect(Collectors.toSet());
Integer[] array = list.stream().toArray(Integer[]::new);
```

Как надо

```
Set<Integer> set = new HashSet<>(list);
Integer[] array = list.toArray(new Integer[0]);
```

Когда Stream не нужен

Как не нужно

```
list.stream().forEach(System.out::println);
list.stream().max(Comparator.naturalOrder()).get();
```

Как надо

```
list.forEach(System.out::println);
Collections.max(list, Comparator.naturalOrder());
```

Полезные ссылки

- **Collection**
  - https://developer.ibm.com/tutorials/j-perry-java-collections/
- **Stream**
  - https://annimon.com/article/2778
- **Streamex**
  - https://github.com/amaembo/streamex

Домашнее задание

Спасибо за атеншен